

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: Задача Коммивояжера.

Студентка гр. 4345

Жилина А. Е.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Задание

В волшебной стране Алгоритмии великий маг, Гамильтон, задумал невероятное путешествие, чтобы связать все города страны закланием процветания. Для этого ему необходимо посетить каждый город ровно один раз, создавая тропу благополучия, и вернуться обратно в столицу, используя минимум своих чародейских сил. Вашей задачей является помощь в прокладывании маршрута с помощью древнего и могущественного алгоритма ветвей и границ.

Карта дорог Алгоритмии перед Гамильтоном представляет собой полный граф, где каждый город соединён магическими порталами с каждым другим. Стоимость использования портала из города в город занимает определённое количество маны, и Гамильтон стремится минимизировать общее потребление магической энергии для закрепления проклятия.

Входные данные:

Первая строка содержит одно целое число N (N — количество городов). Города нумеруются последовательными числами от 0 до $N-1$. Следующие N строк содержат по N чисел каждая, разделённых пробелами, формируя таким образом матрицу стоимостей M . Каждый элемент $M_{i,j}$ этой матрицы представляет собой затраты маны на перемещение из города i в город j .

Выходные данные:

Первая строка: Список из N целых чисел, разделённых пробелами, обозначающих оптимальный порядок городов в магическом маршруте Гамильтона. В начале идёт город 0, с которого начинается маршрут, затем последующие города до тех пор, пока все они не будут посещены. Вторая строка: Число, указывающее на суммарное количество израсходованной маны для завершения пути.

Тестовые данные			^
№ Теста	Входные данные	Выходные данные	
1	3 -1 1 3 3 -1 1 1 2 -1	0 1 2 3.0	
2	4 -1 3 4 1 1 -1 3 4 9 2 -1 4 8 9 2 -1	0 3 2 1 6.0	

Вариант 4. МВиГ: последовательный рост пути + использование для отсечения двух нижних оценок веса оставшегося пути: 1) полусуммы весов двух легчайших рёбер по всем кускам; 2) веса МОД. Эвристика выбора дуги — не в глубину, а по антиприоритету $(S/k+L/N)(4N/(3N+k))$ $(S+L)$. Приближённый алгоритм: АВБГ "улучшенный". Замечание к варианту 4. И МВиГ, и АВБГ "улучшенный" начинать со стартовой вершины.

Описание алгоритма

Данная программа решает задачу коммивояжера с помощью метода ветвей и границ (МВиГ). Основная стратегия заключается в направленном обходе дерева возможных маршрутов с немедленным отсечением тех путей, которые гарантированно хуже уже известного решения. Для принятия решения об отсечении алгоритм постоянно сравнивает верхнюю границу (стоимость текущего лучшего замкнутого маршрута) и нижнюю оценку (оптимистичный прогноз стоимости завершения строящегося пути).

Для получения качественной стартовой верхней границы применяется жадная эвристика — АВБГ «улучшенный». Сначала формируется базовый отрезок из стартового города и ближайшего к нему соседа. Затем на каждой итерации оставшиеся города примеряются на все возможные позиции в уже построенном подцикле. Выбираются такие город и позиция, что вставка дает наименьший прирост общей длины (с учетом разрыва старого ребра и добавления двух новых). Это позволяет задать строгий начальный рекорд (UB), который значительно ускоряет последующие отсечения.

В основном цикле МВиГ незаконченные маршруты извлекаются из приоритетной очереди (min-heap), что позволяет в первую очередь развивать самые перспективные ветви. При переходе в новый город рассчитывается нижняя оценка. Она определяется как максимум из двух метрик: первая (LB1) — округленная вверх полусумма двух наименьших инцидентных ребер для каждого оставшегося компонента пути; вторая (LB2) — вес минимального остовного дерева, которое строится с помощью алгоритма Прима для не посещенных вершин графа и краев оборванного маршрута. Если сумма стоимости пройденного отрезка и вычисленной нижней оценки превышает или равна текущему рекорду, ветвь признается тупиковой и отсекается.

Сложность по времени:

- **Начальный этап:** Считывание матрицы стоимостей требует $O(N^2)$ операций. Работа стартовой эвристики (улучшенной вставки) занимает $O(N^3)$ времени, так как в цикле для каждого из оставшихся городов

(около N итераций) проверяется $O(N)$ кандидатов на вставку и $O(N)$ возможных позиций в текущем маршруте.

- **Основной цикл:** В худшем сценарии (например, при крайне неудачном распределении весов или слабой работе отсечений) алгоритм деградирует до полного перебора, образуя дерево, количество узлов в котором пропорционально $O(N!)$. При генерации каждого нового узла-потомка программа копирует массив текущего пути за $O(N)$ и вычисляет нижнюю оценку L . Процесс оценки, включающий расчет $LB1$ и построение минимального остовного дерева ($LB2$), выполняется за $O(N^2)$. В результате общая асимптотика по времени в худшем случае достигает $O(N! * N^2)$. Однако на практике, благодаря сильной эвристике и жесткой функции оценки, количество исследуемых узлов кардинально сокращается.

Сложность по памяти:

Основной объем памяти на этапе инициализации выделяется под двумерную матрицу графа, что занимает $O(N^2)$. Локальные переменные и вспомогательные массивы для алгоритма Прима (min_e , in_mst) требуют $O(N)$ памяти.

Ключевым потребителем памяти является приоритетная очередь pq , в которой хранятся отложенные узлы дерева решений. В предельном худшем случае максимальная ширина такого дерева, а значит и размер очереди, может достигать $O((N-1)!)$ элементов. Каждый элемент очереди содержит кортеж с отдельным списком (частичным маршрутом) длиной до N , на что расходуется $O(N)$ памяти. Таким образом, пиковое потребление памяти алгоритмом оценивается как $O(N!)$.

Структуры данных и ключевые переменные:

- `cost (list)` — базовая структура данных, представляющая собой двумерный список (матрицу смежности). Хранит стоимости (веса) переходов между всеми парами вершин (городов).

- `N (int)` — переменная, хранящая общее количество вершин (городов) во входном графе.
- `pq (list)` — список, используемый в качестве приоритетной очереди для обхода дерева решений (Best-First Search) с помощью модуля `heapq`. Хранит узлы в виде кортежей формата `(bound, path, S)`, где `bound` — нижняя оценка стоимости, `path` — текущий маршрут, а `S` — накопленная стоимость. Сортировка автоматически происходит по возрастанию поля `bound`.
- `UB (int / float)` — переменная верхней границы (Upper Bound). Хранит стоимость лучшего полного маршрута (рекорда), найденного на данный момент. Используется для отсека неперспективных ветвей графа.
- `USE_PRECISE_ALGORITHM, USE_FILE_INPUT (bool)` — глобальные флаги конфигурации, определяющие режим работы скрипта (запуск точного или приближенного алгоритма) и способ чтения входных данных (из файла или стандартного потока).

Функции алгоритма:

- `def log_to_file(text):` Глобальная утилитарная функция для логирования. Открывает файл `log.txt` в режиме дозаписи и сохраняет текстовую информацию о ходе выполнения алгоритма для последующей отладки.

Аргументы:

- `text` — данные (строка или приводимый к строке объект), которые необходимо записать в лог-файл.
- `def get_improved_nearest_insertion(cost, N):` Эвристическая функция для поиска начального приближенного решения. Формирует стартовый маршрут методом "улучшенной" ближайшей вставки: сначала находит ближайший город к стартовому, а затем последовательно добавляет остальные вершины в те позиции цикла, которые минимизируют прирост общей стоимости пути. Возвращает кортеж из итоговой стоимости и построенного списка вершин.

Аргументы:

- `cost` — матрица стоимостей переходов (весов графа).
- `N` — общее количество городов в графе.
- `def calc_LB1(num_pieces, get_cost):` Вспомогательная функция вычисления первой нижней оценки (Lower Bound 1). Рассчитывает полусумму весов двух легчайших инцидентных ребер (входящего и исходящего) для каждого из свободных компонентов (кусков) маршрута с учетом возможных совпадений.

Аргументы:

- `num_pieces` — количество независимых компонентов (текущий путь + свободные вершины).
- `get_cost` — функция-замыкание, возвращающая стоимость перехода между двумя заданными компонентами.
- `def calc_LB2(num_pieces, get_cost):` Вспомогательная функция вычисления второй нижней оценки (Lower Bound 2). Строит минимальное остовное дерево (МОД) на оставшихся компонентах графа, используя алгоритм Прима. Возвращает суммарный вес ребер построенного дерева.

Аргументы:

- `num_pieces` — количество независимых компонентов.
- `get_cost` — функция-замыкание для определения весов между компонентами.
- `def calc_L(path, cost, N):` Главная функция вычисления нижней оценки стоимости завершения текущего маршрута. Если маршрут неполный, определяет множество непосещенных вершин, инкапсулирует логику получения весов в локальную функцию `get_cost` и вызывает `calc_LB1` и `calc_LB2`. Возвращает максимум из двух полученных оценок для обеспечения наиболее строгого отсечения.

Аргументы:

- `path` — список, содержащий текущий частичный маршрут.

- `cost` — матрица стоимостей переходов.
- `N` — общее количество городов в графе.
- `def solve_branch_and_bound(cost, N):` Основная функция, реализующая метод ветвей и границ (МВиГ). Вычисляет начальный рекорд с помощью эвристики, инициализирует приоритетную очередь и циклично извлекает наиболее перспективные узлы. Генерирует новые ветви графа, вычисляет для них оценки `new_bound` и отсекает те переходы, оценка которых превышает текущий рекорд `UB`. Возвращает кортеж из оптимальной стоимости и итогового маршрута.

Аргументы:

- `cost` — матрица стоимостей переходов.
- `N` — общее количество городов в графе.
- `def main():` Точка входа в программу. Отвечает за предварительную очистку файла логов, парсинг входных данных (формирование размерности `N` и матрицы `cost`) из файла `matrix.txt` или через `input()`. В зависимости от установленных констант вызывает точный (`solve_branch_and_bound`) или приближенный (`get_improved_nearest_insertion`) алгоритм, после чего форматирует и выводит результат в консоль. Не принимает аргументов.

Тестирование

Результаты тестирования представлены в табл. 1 и 2.

Таблица 1 – Результаты тестирования (Алгоритм МВиГ)

№ п/п	Входные данные	Выходные данные	Комментарии
1.	4 0 7 4 5 5 0 3 6 5 4 0 5 2 2 6 0	0 3 1 2 15.0	Верный результат
2.	3 0 1 3 1 0 2 3 1 0	0 2 1 5.0	Верный результат
3.	5 0 35 33 5 22 35 0 44 13 38 33 44 0 41 41 5 13 41 0 33 22 38 41 33 0	0 3 1 2 4 125.0	Верный результат
4.	7 0 21 1 2 31 10 9 13 0 1 20 37 38 35 34 27 0 15 27 19 17 21 7 16 0 15 23 24 16 39 32 20 0 27 39 9 22 8 26 25 0 12 38 32 28 22 24 4 0	0 3 1 2 6 5 4 72.0	Верный результат

Таблица 2 – Результаты тестирования (Алгоритм АВБГ “улучшенный”)

№ п/п	Входные данные	Выходные данные	Комментарии
1.	4 0 7 4 5 5 0 3 6 5 4 0 5 2 2 6 0	0 2 1 3 16.0	Верный результат (с учётом того, что алгоритм даёт приближенный ответ)
2.	3 0 1 3 1 0 2 3 1 0	0 2 1 5.0	Верный результат
3.	5	0 4 2 1 3	Верный результат

	0 35 33 5 22 35 0 44 13 38 33 44 0 41 41 5 13 41 0 33 22 38 41 33 0	125.0	(алгоритм нашёл маршрут равный по длине алгоритму МВиГ, однако лексикографически путь больше)
4.	7 0 21 1 2 31 10 9 13 0 1 20 37 38 35 34 27 0 15 27 19 17 21 7 16 0 15 23 24 16 39 32 20 0 27 39 9 22 8 26 25 0 12 38 32 28 22 24 4 0	0 3 1 2 6 5 4 72.0	Верный результат

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.py

```
import heapq

USE_PRECISE_ALGORITHM = False
USE_FILE_INPUT = False

def log_to_file(text):
    with open("log.txt", "a", encoding="utf-8") as f:
        f.write(str(text) + "\n")

# Функция для поиска приближенного решения (АВБГ "улучшенный")
def get_improved_nearest_insertion(cost, N):
    log_to_file(f"\n[АВБГ] Запуск алгоритма поиска приближенного решения, N={N}")
    if N <= 1:
        log_to_file("[АВБГ] N <= 1, возвращаем 0, [0]")
        return 0, [0]

    # Инициализация начального маршрута и массива посещений
    path = [0]
    vis = [False] * N
    vis[0] = True
    log_to_file("[АВБГ] Инициализация: начальный путь=[0]")

    # Шаг 1: Формируем стартовый цикл из двух вершин
    best_k = -1
    min_c = float('inf')

    # Перебираем все города для поиска ближайшего к стартовому
    log_to_file("[АВБГ] Шаг 1: Поиск ближайшего города к стартовому (0)")
    for k in range(1, N):
        c = cost[0][k] + cost[k][0]
        log_to_file(
            f"[АВБГ] Проверка города {k}: c = cost[0][k] + cost[k][0] = {c}"
        )
        if c < min_c:
            log_to_file(f"[АВБГ] --> Найдено новое минимальное c: {c} < {min_c}. Выбираем {k}")
            min_c = c
            best_k = k

    # Добавляем найденный город в начальный маршрут
    path.append(best_k)
    vis[best_k] = True
    log_to_file(f"[АВБГ] В начальный маршрут добавлен город {best_k}. Текущий путь: {path}")

    # Шаг 2: Вставляем остальные вершины, минимизируя прирост стоимости
    log_to_file("[АВБГ] Шаг 2: Вставка остальных вершин")
    for _ in range(2, N):
        best_k = -1
        best_insert_idx = -1
```

```

min_delta = float('inf')
log_to_file(f"[АВВГ] Итерация вставки. Текущий путь: {path}")

# Пропускаем города, которые уже включены в цикл
for k in range(N):
    if vis[k]:
        continue

    log_to_file(f"[АВВГ] Оценка вставки непосещенного города
{k}:")

    # Проверяем каждую позицию в текущем цикле для вставки
    for i in range(len(path)):
        u = path[i]
        v = path[(i + 1) % len(path)]
        delta = cost[u][k] + cost[k][v] - cost[u][v]
        log_to_file(
            f"[АВВГ] Позиция между {u} и {v}: delta =
cost[{u}][{k}] + cost[{k}][{v}] - cost[{u}][{v}] = {cost[u][k]} +
{cost[k][v]} - {cost[u][v]} = {delta}")

        # Сохраняем лучшую позицию, дающую минимальный прирост
пути
        if delta < min_delta:
            log_to_file(f"[АВВГ] --> Найдена лучшая позиция!
min_delta обновлена: {min_delta} -> {delta}")
            min_delta = delta
            best_k = k
            best_insert_idx = i + 1

    # Вставляем выбранный город в оптимальную позицию маршрута
    path.insert(best_insert_idx, best_k)
    vis[best_k] = True
    log_to_file(f"[АВВГ] Город {best_k} вставлен на позицию
{best_insert_idx}. Новый путь: {path}")

# Подсчет финальной стоимости построенного пути
total_cost = 0
log_to_file("[АВВГ] Подсчет финальной стоимости построенного пути:")
for i in range(len(path)):
    u = path[i]
    v = path[(i + 1) % len(path)]
    w = cost[u][v]
    log_to_file(f"[АВВГ] Ребро {u} -> {v}, вес {w}. total_cost =
{total_cost} + {w} = {total_cost + w}")
    total_cost += w

    log_to_file(f"[АВВГ] Итог АВВГ: стоимость = {total_cost}, путь =
{path}\n")
    return total_cost, path

# Оценка 1: Полусумма весов двух легчайших ребер для каждого куска
def calc_LB1(num_pieces, get_cost):
    log_to_file(f"[LB1] Запуск вычисления LB1 для {num_pieces} кусков")
    lb1_sum = 0

    for i in range(num_pieces):
        log_to_file(f"[LB1] Обработка куска i={i}:")

```

```

min_in, min2_in, arg_in = float('inf'), float('inf'), -1
min_out, min2_out, arg_out = float('inf'), float('inf'), -1

# Поиск минимального и второго минимального входящего ребра
for j in range(num_pieces):
    if i == j: continue

    c_in = get_cost(j, i)
    log_to_file(f"[LB1]      Проверка входящего ребра {j} -> {i}:
вес {c_in}")
    if c_in < min_in:
        min2_in = min_in
        min_in = c_in
        arg_in = j
        log_to_file(f"[LB1]      Обновление: min_in={min_in}
(из {arg_in}), min2_in={min2_in}")
    elif c_in < min2_in:
        min2_in = c_in
        log_to_file(f"[LB1]      Обновление: min2_in={min2_in}")

    # Поиск минимального и второго минимального исходящего ребра
    c_out = get_cost(i, j)
    log_to_file(f"[LB1]      Проверка исходящего ребра {i} -> {j}:
вес {c_out}")
    if c_out < min_out:
        min2_out = min_out
        min_out = c_out
        arg_out = j
        log_to_file(f"[LB1]      Обновление: min_out={min_out}
(в {arg_out}), min2_out={min2_out}")
    elif c_out < min2_out:
        min2_out = c_out
        log_to_file(f"[LB1]      Обновление:
min2_out={min2_out}")

    log_to_file(
        f"[LB1]      Итоги для куска {i}: min_in={min_in} (от {arg_in}),
min2_in={min2_in} | min_out={min_out} (к {arg_out}),
min2_out={min2_out}")
    # Прибавление весов с учетом возможного совпадения компонентов
    if arg_in == arg_out and arg_in != -1:
        val1 = min_in + min2_out
        val2 = min2_in + min_out
        added_val = min(val1, val2)
        log_to_file(
            f"[LB1]      Совпадение arg_in и arg_out ({arg_in}).
Считаем: min({min_in} + {min2_out}, {min2_in} + {min_out}) = min({val1},
{val2}) = {added_val}")
        lb1_sum += added_val
    else:
        added_val = min_in + min_out
        log_to_file(f"[LB1]      Разные источники. Считаем: min_in +
min_out = {min_in} + {min_out} = {added_val}")
        lb1_sum += added_val

    log_to_file(f"[LB1]      Текущая сумма lb1_sum = {lb1_sum}")

# Округление полусуммы весов вверх

```

```

        result = (lb1_sum + 1) // 2
        log_to_file(f"[LB1] Итоговая сумма {lb1_sum}. Возвращаем ({lb1_sum}
+ 1) // 2 = {result}")
        return result

# Оценка 2: Минимальное Остовное Дерево (МОД) на кусках
def calc_LB2(num_pieces, get_cost):
    log_to_file(f"[LB2] Запуск вычисления LB2 (МОД) для {num_pieces}
кусков")
    LB2 = 0

    # Подготовка структур для алгоритма Прима (построение МОД)
    min_e = [float('inf')] * num_pieces
    in_mst = [False] * num_pieces
    min_e[0] = 0

    # Выбор непосещенной вершины с минимальным весом присоединения
    for _ in range(num_pieces):
        v = -1
        for j in range(num_pieces):
            if not in_mst[j] and (v == -1 or min_e[j] < min_e[v]):
                v = j

        if min_e[v] == float('inf'):
            log_to_file("[LB2] Обнаружена недостижимая вершина (inf),
прерывание цикла МОД")
            break

        # Пометка вершины как посещенной и добавление веса в оценку
        in_mst[v] = True
        old_LB2 = LB2
        LB2 += min_e[v]
        log_to_file(f"[LB2] Выбрана вершина v={v} с мин. ребром
{min_e[v]}. LB2 = {old_LB2} + {min_e[v]} = {LB2}")

        # Обновление минимальных расстояний от дерева до оставшихся
        вершин
        for to in range(num_pieces):
            if not in_mst[to]:
                c = min(get_cost(v, to), get_cost(to, v))
                if c < min_e[to]:
                    log_to_file(f"[LB2] Обновление расстояния до
{to}: min_e[{to}] снижено с {min_e[to]} до {c}")
                    min_e[to] = c

        log_to_file(f"[LB2] Итоговое значение LB2 = {LB2}")
        return LB2

# Главная функция вычисления нижней оценки L
def calc_L(path, cost, N):
    log_to_file(f"\n[L] Вычисление L для пути: {path}")
    # Если маршрут уже полный, возвращаем стоимость замыкающего ребра
    if len(path) == N:
        close_edge = cost[path[-1]][path[0]]
        log_to_file(

```

```

        f"[L] Маршрут полный. Замыкающее ребро {path[-1]} ->
{path[0]} имеет вес {close_edge}. Возвращаем {close_edge}."
        return close_edge

    # Для предпоследнего шага просто добавляем единственный оставшийся
    город
    if len(path) == N - 1:
        vis = set(path)
        missing = -1
        for i in range(N):
            if i not in vis:
                missing = i
                break
        c1 = cost[path[-1]][missing]
        c2 = cost[missing][path[0]]
        res = c1 + c2
        log_to_file(
            f"[L] Маршрут без одной вершины ({missing}). Возвращаем
{path[-1]}->{missing} ({c1}) + {missing}->{path[0]} ({c2}) = {res}")
        return res

    # Формируем список еще не посещенных вершин (свободных кусков)
    vis = set(path)
    U = [i for i in range(N) if i not in vis]
    log_to_file(f"[L] Свободные вершины (U): {U}")

    # Количество независимых компонентов (текущий путь + свободные
    вершины)
    num_pieces = len(U) + 1
    log_to_file(f"[L] Количество независимых компонентов: {num_pieces}")

    # Функция определения расстояний между различными компонентами графа
    def get_cost(a, b):
        if a == b: return float('inf')
        if a == 0: return cost[path[-1]][U[b - 1]]
        if b == 0: return cost[U[a - 1]][path[0]]
        return cost[U[a - 1]][U[b - 1]]

    # Вычисление двух оценок с помощью вынесенных функций
    LB1 = calc_LB1(num_pieces, get_cost)
    LB2 = calc_LB2(num_pieces, get_cost)

    # Возврат наибольшей из двух нижних оценок
    final_L = max(LB1, LB2)
    log_to_file(f"[L] max(LB1={LB1}, LB2={LB2}) = {final_L}. Возвращаем
{final_L}")
    return final_L

# Функция метода ветвей и границ (МВИГ)
def solve_branch_and_bound(cost, N):
    # Инициализируем рекорд приближенным решением
    UB, best_path = get_improved_nearest_insertion(cost, N)

    log_to_file("\n=== ЗАПУСК МЕТОДА ВЕТВЕЙ И ГРАНИЦ ===")
    log_to_file(f"[МВИГ] Начальный UB (рекорд) = {UB}, начальный лучший
путь = {best_path}")

```

```

# Приоритетная очередь
pq = []

# Вычисление оценки для стартового города и инициализация очереди
initial_L = calc_L([0], cost, N)
log_to_file(f"[МВиГ] Стартовая оценка initial_L = {initial_L}")

# Формат кортежа: (bound, path, S).
# Обеспечивает сортировку по bound, затем лексикографически по пути
heapq.heappush(pq, (initial_L, [0], 0))
log_to_file(f"[МВиГ] В очередь помещен стартовый узел:
bound={initial_L}, path=[0], S=0")

iteration_count = 0
# Основной цикл метода ветвей и границ
while pq:
    iteration_count += 1
    log_to_file(f"\n[МВиГ] --- Итерация {iteration_count} ---")
    log_to_file(f"[МВиГ] Длина очереди (heapq): {len(pq)}")

    # Вывод первых 5 элементов очереди
    top_5 = pq[:5]
    log_to_file(f"[МВиГ] Первые элементы очереди:")
    for idx, item in enumerate(top_5):
        log_to_file(f"                {idx + 1}: bound={item[0]},
path={item[1]}, S={item[2]}")

    # Извлечение самого перспективного узла из очереди
    curr_bound, curr_path, curr_S = heapq.heappop(pq)
    log_to_file(f"[МВиГ] Извлечен узел: bound={curr_bound},
path={curr_path}, текущая стоимость S={curr_S}")

    # Отсечение ветви: если оценка хуже рекорда, дальше не идем
    if curr_bound > UB:
        log_to_file(f"[МВиГ] Отсечение! curr_bound ({curr_bound}) >
UB ({UB}). Завершаем или пропускаем ветвь.")
        break

    # Если все города посещены, проверяем замыкание маршрута
    if len(curr_path) == N:
        close_cost = cost[curr_path[-1]][curr_path[0]]
        full_cost = curr_S + close_cost
        log_to_file(
            f"[МВиГ] Найден полный маршрут. full_cost = S ({curr_S})
+ замыкание {curr_path[-1]}->{curr_path[0]} ({close_cost}) =
{full_cost}")

        # Проверка на улучшение текущего рекорда
        if full_cost < UB:
            log_to_file(
                f"[МВиГ] !!! Обновление рекорда !!! Старый UB={UB},
новый UB={full_cost}. Новый лучший путь: {curr_path}")
            UB = full_cost
            best_path = curr_path

        # Сохранение лексикографически меньшего пути при равной
стоимости
        elif full_cost == UB and curr_path < best_path:

```

```

        log_to_file(
            f"[МВиГ] Стоимость равна текущему рекорду ({UB}),
но путь лексикографически меньше. Обновляем лучший путь: {curr_path}")
        best_path = curr_path
    else:
        log_to_file(f"[МВиГ] Маршрут не улучшает текущий рекорд
({UB}).")
        continue

    # Подготовка множества посещенных вершин для текущего узла
    vis = set(curr_path)
    last_v = curr_path[-1]
    log_to_file(f"[МВиГ] Ветвление из вершины {last_v}. Посещенные
вершины: {vis}")

    # Генерация новых ветвей для каждого доступного города
    for nxt in range(N):
        if nxt not in vis:
            log_to_file(f"[МВиГ] Рассматриваем переход {last_v} ->
{nxt}:")

            # Формирование нового пути, расчет стоимости и оценки L
            edge_cost = cost[last_v][nxt]
            new_S = curr_S + edge_cost
            log_to_file(f"[МВиГ] Новая стоимость new_S = {curr_S}
+ {edge_cost} = {new_S}")

            new_path = curr_path + [nxt]
            new_L = calc_L(new_path, cost, N)
            new_bound = new_S + new_L
            log_to_file(
                f"[МВиГ] Оценка для {new_path}: new_bound = new_S
({new_S}) + new_L ({new_L}) = {new_bound}")

            # Добавляем только если есть шанс улучшить рекорд
            if new_bound <= UB:
                log_to_file(f"[МВиГ] --> Успешно! {new_bound}
<= UB ({UB}). Добавляем в очередь.")
                heapq.heappush(pq, (new_bound, new_path, new_S))
            else:
                log_to_file(f"[МВиГ] --> Отсев! {new_bound} >
UB ({UB}). Переход проигнорирован.")

        log_to_file(f"\n=== ЗАВЕРШЕНИЕ МВиГ. Итоговый UB={UB},
путь={best_path} ===")
        # Возврат оптимальной стоимости и маршрута
        return UB, best_path

def main():
    # Инициализация (очистка) файла логов
    with open("log.txt", "w", encoding="utf-8") as f:
        f.write("\n")

    if USE_FILE_INPUT:
        with open("matrix.txt", "r", encoding="utf-8") as f:
            lines = f.read().split()
            if not lines:
                log_to_file("Ошибка: файл пуст")

```

```

        return
    N = int(lines[0])
    values = [int(x) for x in lines[1:]]
    cost = [values[i * N : (i + 1) * N] for i in range(N)]
else:
    # Считывание всех входных данных из стандартного потока
    N = int(input())
    cost = [[int(i) for i in input().split()] for _ in range(N)]

log_to_file(f"Считано {N}")
log_to_file(f"Матрица стоимостей: {cost}")

if USE_PRECISE_ALGORITHM:
    # Запуск метода ветвей и границ
    UB, best_path = solve_branch_and_bound(cost, N)
else:
    # Запуск АББГ "улучшенный"
    UB, best_path = get_improved_nearest_insertion(cost, N)

# Форматированный вывод оптимального маршрута и его стоимости
print(" ".join(map(str, best_path)))
print(f"{UB}.0")

if __name__ == '__main__':
    main()

```

Название файла: generate.py

```

import random

def main():
    n = int(input("Размер матрицы N: "))
    is_symmetric = input("Сделать матрицу симметричной? (y/n): ")
    is_symmetric = is_symmetric.strip().lower()[0] == "y"

    matrix = [[0] * n for _ in range(n)]

    for i in range(n):
        for j in range(n):
            if i == j:
                matrix[i][j] = 0
            elif is_symmetric and i > j:
                matrix[i][j] = matrix[j][i]
            elif i < j or not is_symmetric:
                matrix[i][j] = random.randint(1, 100)

    filename = "matrix.txt"
    with open(filename, "w", encoding="utf-8") as f:
        f.write(f"{n}\n")
        for row in matrix:
            f.write(" ".join(map(str, row)) + "\n")

    print(f"Матрица успешно сгенерирована и сохранена в {filename}.")

if __name__ == '__main__':
    main()

```